

Extremal Paths in Anti-de Sitter Space: Progress Report

Liviru

October 2026

1 Introduction

This report follows an investigation into piecewise extremal paths between two points in anti-de Sitter (AdS) space, with the goal being to find the optimised path between two points when the path is built from two straight pieces in the conformal time-space plane.

2 Set-up: Distance Landscapes

Before working in AdS we wanted a setting where we already have some intuition, so that mistakes in the machinery would be easy to spot. A Mercator map of a sphere is a good candidate: a straight line drawn on the map is not the shortest path on the sphere, which is the same situation as a “straight” piecewise path in the conformal plane versus the true geodesic. With the mercator’s metric, we decided to map the distance from a fixed origin to every point of the canvas as a heatmap, first for the Mercator metric and then for the AdS metric, to see what the landscape of path lengths looks like before asking where its extrema are. In the AdS case the interval vanishes along the 45° directions, which are null.

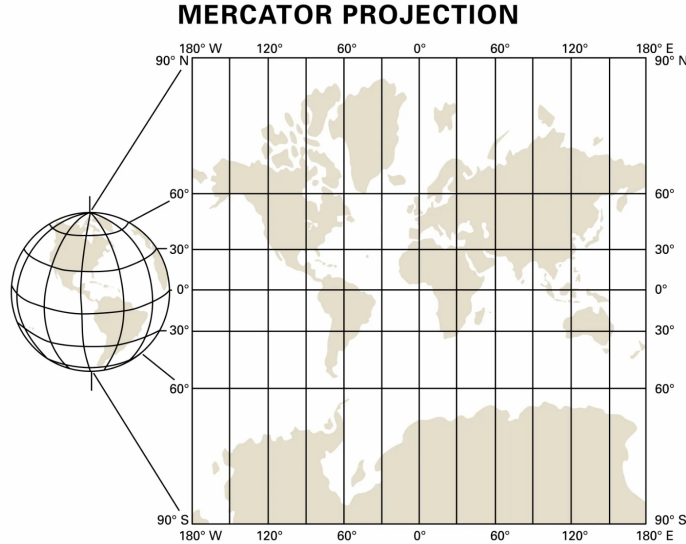


Figure 1: Mercator Projection of the Earth

Mercator warm-up

We first found just the straight line-distance on a Mercator canvas, integrated numerically, between two points.

Listing 1: Mercator distance (code/01_mercator.py).

```
1 import math
2
3 def mercator_line_length_numerical(x1, y1, x2, y2, W_world, R=6371.0, steps
=10000):
4     """Real-world length of a straight line drawn on a Mercator canvas."""
5     total_distance = 0.0
6     dt = 1.0 / steps
7     dx_dt = x2 - x1
8     dy_dt = y2 - y1
9     for i in range(steps):
10         t = (i + 0.5) * dt
11         y = y1 + t * dy_dt
12         y_scaled = (2.0 * math.pi * y) / W_world
13         # Convert Mercator y to latitude phi
14         phi = 2.0 * math.atan(math.exp(y_scaled)) - (math.pi / 2.0)
15         dlambda_dt = (2.0 * math.pi * dx_dt) / W_world
16         dphi_dt = ((2.0 * math.pi * dy_dt) / W_world) * math.cos(phi)
17         ds_squared = (R**2 * dphi_dt**2) + (R**2 * (math.cos(phi)**2) *
            dlambda_dt**2)
18         total_distance += math.sqrt(ds_squared) * dt
19     return total_distance
20
21
22 def two_segment_mercator_distance(x1, y1, x2, y2, theta_degrees, W_world,
23                                   R=6371.0, steps_per_seg=5000):
24     """Climb at angle theta until y = y2, then run horizontally to x2."""
25     theta_rad = math.radians(theta_degrees)
26     if math.isclose(math.sin(theta_rad), 0.0):
27         raise ValueError("Angle theta cannot be 0 or 180 degrees for a
            vertical climb.")
28     corner_x = x1 + (y2 - y1) / math.tan(theta_rad)
29     corner_y = y2
30     dist1 = mercator_line_length_numerical(x1, y1, corner_x, corner_y,
        W_world, R, steps_per_seg)
31     dist2 = mercator_line_length_numerical(corner_x, corner_y, x2, y2,
        W_world, R, steps_per_seg)
32     return dist1 + dist2, corner_x, corner_y
33
34
35 # canvas_width = 2000, start = (200, 150), end = (900, 600)
36 # straight line -> 9598.71 km; 45 degree climb then horizontal -> 8825.84 km
```

If we repeat this for every point in the canvas, we end up with a heatmap:

Listing 2: Mercator distance heatmap (code/02_mercator_heatmap.py).

```

1 import numpy as np
2
3 def generate_distance_heatmap(orig_x, orig_y, W_world, H_world, R=6371.0,
4                               resolution=80):
5     """Distance from a fixed origin to every point of the Mercator canvas."""
6     x_coords = np.linspace(0, W_world, resolution)
7     y_coords = np.linspace(-H_world / 2, H_world / 2, resolution)
8     heatmap = np.zeros((resolution, resolution))
9     for i, y in enumerate(y_coords):
10         for j, x in enumerate(x_coords):
11             heatmap[i, j] = mercator_line_length_numerical(orig_x, orig_y, x,
12                                                             y, W_world, R)
13     return x_coords, y_coords, heatmap
14
15 # canvas 2000 x 2000, R = 6371, origin (0, 0), resolution 100

```

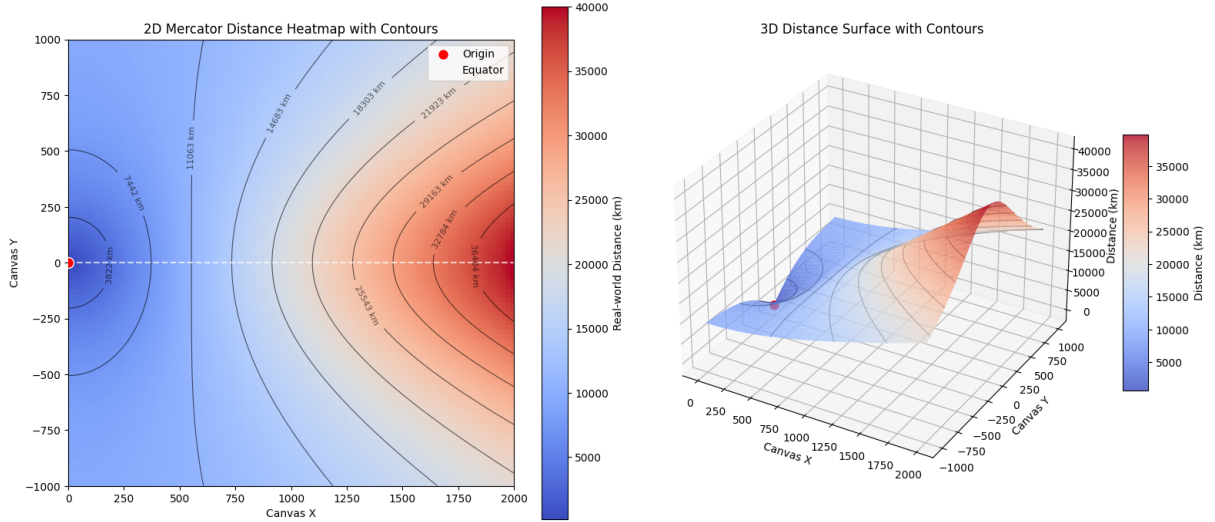


Figure 2: Mercator distance from the origin as a 2D heatmap with contours and a 3D surface.

Note the sinusoidal behaviour, and how points that are 'further away' are not necessarily further away. If we repeat this method on the same grid for the AdS metric, from the origin $(-3, 0)$. Timelike steps return imaginary distances, so only spacelike points are drawn.

Listing 3: AdS heatmap, spacelike points only (code/03_ads_heatmap_spacelike.py).

```

1 import math
2 import numpy as np
3
4 def integrate_ads_segment(xa, ya, xb, yb, Lambda=1.0, steps=100):
5     total_interval = 0.0
6     dt = 1.0 / steps
7     dx_dt = xb - xa
8     dy_dt = yb - ya
9     for i in range(steps):
10         t = (i + 0.5) * dt
11         y = ya + t * dy_dt # y represents eta
12         cos_y = math.cos(y)
13         if math.isclose(cos_y, 0.0, abs_tol=1e-5):
14             continue
15         conformal_factor = 1.0 / ((Lambda**2) * (cos_y**2))
16         ds_squared = conformal_factor * (-(dy_dt**2) + (dx_dt**2))
17         if ds_squared < 0: # timelike: leave blank
18             return np.nan
19         total_interval += math.sqrt(ds_squared) * dt
20     return total_interval
21
22
23 def generate_ads_heatmap(orig_x, orig_y, Lambda=1.0, x_lim=3.0, resolution
24 =80):
25     x_coords = np.linspace(-x_lim, x_lim, resolution)
26     margin = 0.05
27     y_coords = np.linspace(-math.pi / 2 + margin, math.pi / 2 - margin,
28 resolution)
29     heatmap = np.full((resolution, resolution), np.nan)
30     for i, y in enumerate(y_coords):
31         for j, x in enumerate(x_coords):
32             heatmap[i, j] = integrate_ads_segment(orig_x, orig_y, x, y,
33 Lambda=Lambda)
34     return x_coords, y_coords, heatmap
35
36 # generate_ads_heatmap(orig_x=-3.0, orig_y=0.0, Lambda=1.0, x_lim=3.0,
37 resolution=80)

```

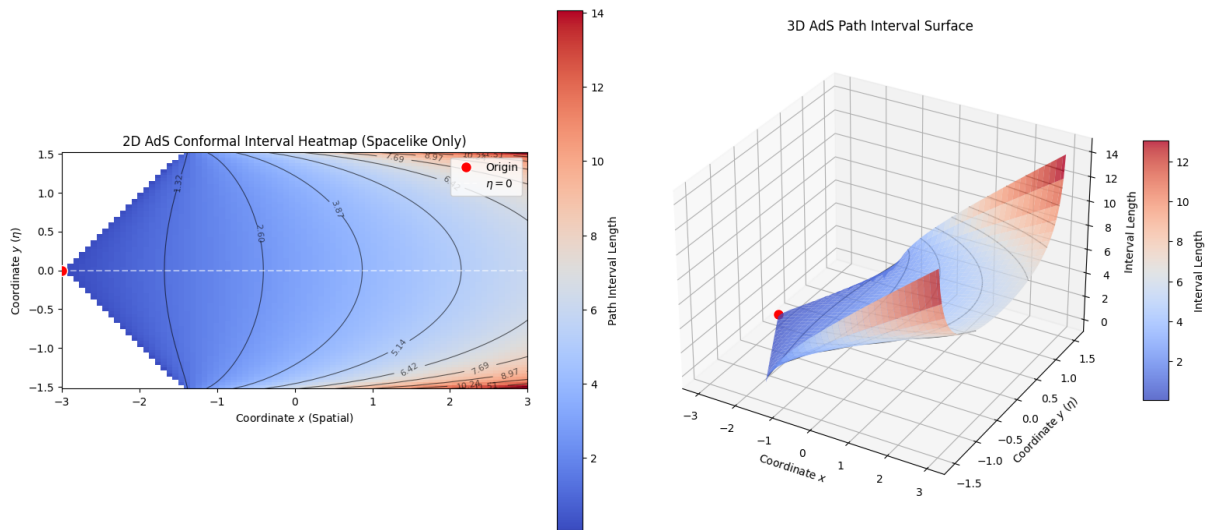


Figure 3: The same heatmap built from the AdS interval (origin at $(-3, 0)$), keeping only spacelike separations.

3 Numerical work

3.1 One-parameter problem: the climb angle

We now began to try and use this technique to optimise paths between points. We started with the easiest path we could think off, a path made of a diagonal segment followed by a horizontal one. We then attempted to find the diagonal angle θ that optimises the total interval. With the start at $(0,0)$ and the end at $(3.0,0.8)$, a first trial at $\theta = 60^\circ$ gives a total interval of 4.3796. Handing the same problem to a bounded scalar optimiser (SciPy `minimize_scalar`) gives $\theta = 45.00^\circ$ with total interval 3.1579.

An answer as tidy as 45° is worth questioning before building on it, so we asked why it comes out. A 45° segment is null in this metric, so it contributes zero interval, and the whole value comes from the horizontal run, $2.2/\cos 0.8 \approx 3.158$. In other words, 45° always came out as optimal for this rigid set-up because of how the interval is computed (with an absolute value, as the code below shows), not because it is a genuine extremum of the physical path. This tells us two things: the absolute value needs to go, and a single angle is too restrictive a family of paths. The second point suggests the next step, which is to free the corner of the path altogether.

The first version of the integrator, which takes the absolute value of ds^2 , is below; we return to it in Section 3.3.

Listing 4: First AdS experiment: climb-angle optimiser (code/04_ads_angle_optimiser.py).

```

1 import math
2 from scipy.optimize import minimize_scalar
3
4 def integrate_ads_segment(xa, ya, xb, yb, Lambda=1.0, steps=5000):
5     """Integrates a path segment in the AdS conformal plane (first version)."""
6
7     total_interval = 0.0
8     dt = 1.0 / steps
9     dx_dt = xb - xa
10    dy_dt = yb - ya
11    for i in range(steps):
12        t = (i + 0.5) * dt
13        y = ya + t * dy_dt # y represents eta
14        cos_y = math.cos(y)
15        if math.isclose(cos_y, 0.0) or abs(y) >= math.pi / 2:
16            return float('inf') # keep the optimiser away from the boundary
17        # Conformal factor: 1 / (Lambda^2 * cos^2(eta))
18        conformal_factor = 1.0 / ((Lambda**2) * (cos_y**2))
19        # ds^2 = conformal_factor * (-deta^2 + drho^2)
20        ds_squared = conformal_factor * (-(dy_dt**2) + (dx_dt**2))
21        # Absolute value: this is what let timelike steps count as distance
22        total_interval += math.sqrt(abs(ds_squared)) * dt
23    return total_interval
24
25 def ads_two_segment_path(x1, y1, x2, y2, theta_degrees, Lambda=1.0):
26     theta_rad = math.radians(theta_degrees)
27     if math.isclose(math.sin(theta_rad), 0.0):
28         return float('inf'), x1, y1
29     corner_x = x1 + (y2 - y1) / math.tan(theta_rad)
30     corner_y = y2
31     dist1 = integrate_ads_segment(x1, y1, corner_x, corner_y, Lambda)
32     dist2 = integrate_ads_segment(corner_x, corner_y, x2, y2, Lambda)
33     return dist1 + dist2, corner_x, corner_y
34
35
36 start_rho, start_eta = 0, 0
37 end_rho, end_eta = 3.0, 0.8
38
39 def find_distance_for_angle(theta):
40     dist, _, _ = ads_two_segment_path(start_rho, start_eta, end_rho, end_eta,
41                                     theta, 1.0)
42     return dist
43
44 climb_angle = minimize_scalar(find_distance_for_angle, bounds=(5.0, 175.0),
45                               method='bounded').x
46 # theta = 60 degrees -> 4.3796 (Figure 5); optimiser -> 45.00 degrees, 3.1579
47 # (Figure 6)

```

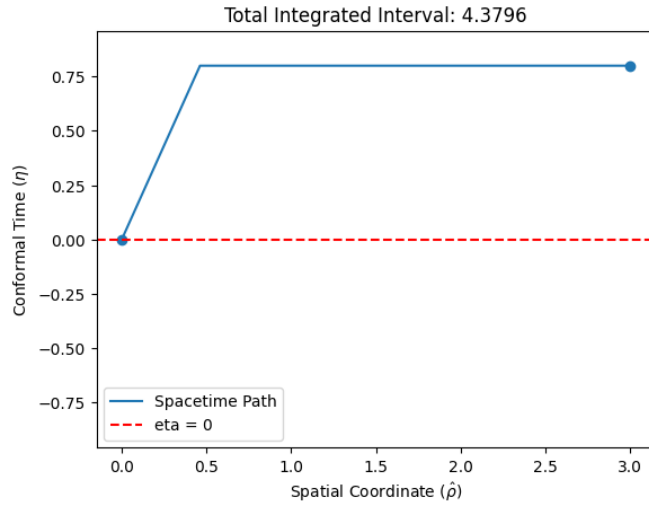


Figure 4: Climb angle of 60° : total interval 4.3796.

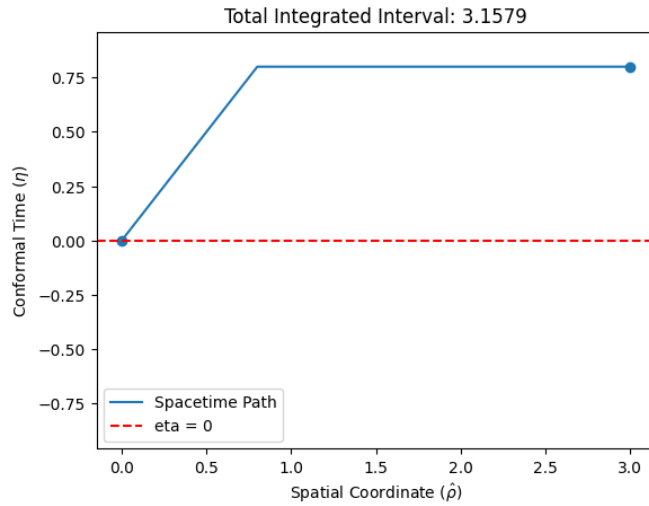


Figure 5: Optimiser result, 45.00° : total interval 3.1579.

3.2 Two-dimensional problem: the inflection point

If the corner is not forced to sit at the end point's height, the path has two free parameters instead of one. The natural next step is therefore to make the inflection point itself the input. We first rewrote the code to accept an inflection point directly, which reproduces the example in Figure 6:

Listing 5: Inflection point supplied directly (code/05_ads_inflection_direct.py).

```
1 def ads_two_segment_path_from_point(x1, y1, x2, y2, xi, yi, Lambda=1.0):
2     """Two-segment path with the inflection point (xi, yi) given directly."""
3     dist1 = integrate_ads_segment(x1, y1, xi, yi, Lambda)
4     dist2 = integrate_ads_segment(xi, yi, x2, y2, Lambda)
5     return dist1 + dist2, xi, yi
6
7
8 # (0, 0) -> (1.0, 0.5) -> (3.0, 0.8) gives a total interval of 3.4088
```

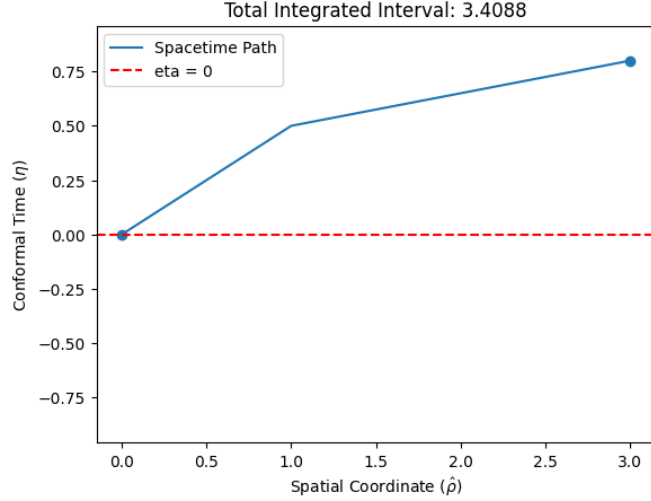


Figure 6: Test inflection point (1.0,0.5) between (0,0) and (3.0,0.8): total interval 3.4088.

Once the inflection point is an input, we can evaluate the total interval for every candidate point on a grid, which gives a heatmap and a 3D surface of the total path length. The hope was that the extremum of this surface would sit where the piecewise path best approximates the geodesic. For one choice of points the first results looked plausible, but for the pair $(0,0)$ to $(\pi/2, \pi/4)$ the minimum landed at $(0.3913, -0.3916)$ with a value of only 0.0996, which makes no sense for a path whose coordinate length is well over one. This is the same warning sign as the 45° result, and it has the same cause: the absolute value on ds^2 lets timelike (imaginary) intervals be counted as real distances, so the search was effectively allowed to use superluminal paths. That minimum sits where both segments are close to null, so the interval there is close to zero.

Listing 6: First grid search: no causal filter, absolute-value integrator, minimum (code/06_ads_grid_unfiltered.py).

```

1 import math
2 import numpy as np
3
4 def generate_ads_inflection_heatmap(start_x, start_y, end_x, end_y, Lambda
    =1.0,
5                                     x_lim=3.0, resolution=80):
6     x_coords = np.linspace(-x_lim, x_lim, resolution)
7     margin = 0.05
8     y_coords = np.linspace(-math.pi / 2 + margin, math.pi / 2 - margin,
9                             resolution)
9     heatmap = np.zeros((resolution, resolution))
10    for i, y in enumerate(y_coords):
11        for j, x in enumerate(x_coords):
12            dist1 = integrate_ads_segment(start_x, start_y, x, y, Lambda=
13                                         Lambda)
14            dist2 = integrate_ads_segment(x, y, end_x, end_y, Lambda=Lambda)
15            heatmap[i, j] = dist1 + dist2
16    return x_coords, y_coords, heatmap
17
18 x_vals, y_vals, dist_grid = generate_ads_inflection_heatmap(
19     0, 0, np.pi / 2, np.pi / 4, Lambda=1.0, x_lim=3.0, resolution=300)
20
21 min_idx = np.unravel_index(np.argmin(dist_grid), dist_grid.shape)
22 opt_y_grid = y_vals[min_idx[0]]
23 opt_x_grid = x_vals[min_idx[1]]
24 min_dist_grid = dist_grid[min_idx]
25 # (0.3913, -0.3916), minimum 0.0996

```

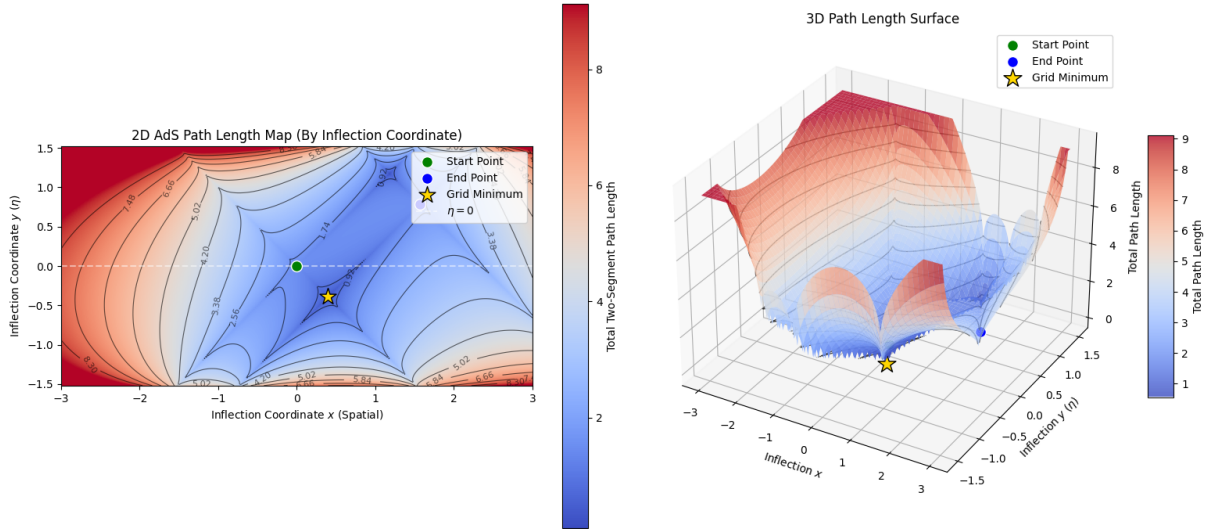


Figure 7: Before the fix: inflection-point heatmap for $(0,0)$ to $(\pi/2, \pi/4)$ with the absolute value in place. The minimum, $(0.3913, -0.3916)$, sits where both segments are almost null, so the interval is close to zero.

3.3 Fix: causal filtering and maximisation

Two changes follow directly from that diagnosis. First, remove the absolute value, so that a timelike step is no longer turned into a real length. Second, because the square root now fails for timelike steps, only evaluate candidate corners that can be reached from the start point and can reach the end point within their 45° causal cones. With the unphysical region gone, the surface

no longer has a trough in an odd place; it has a clear peak inside the causal region, so we now search for the maximum.

Listing 7: Final integrator (code/07_ads_integrator_fixed.py).

```

1 import math
2
3 def integrate_ads_segment(xa, ya, xb, yb, Lambda=1.0, steps=100):
4     """Integrates the interval along a straight segment in the conformal AdS
5         plane."""
6     total_interval = 0.0
7     dt = 1.0 / steps
8     dx_dt = xb - xa
9     dy_dt = yb - ya
10    for i in range(steps):
11        t = (i + 0.5) * dt
12        y = ya + t * dy_dt # y represents eta
13        cos_y = math.cos(y)
14        if math.isclose(cos_y, 0.0, abs_tol=1e-5) or abs(y) >= math.pi / 2:
15            return float('inf') # boundary protection
16        # Conformal factor: 1 / (Lambda^2 * cos^2(eta))
17        conformal_factor = 1.0 / ((Lambda**2) * (cos_y**2))
18        # ds^2 = conformal_factor * (-deta^2 + drho^2)
19        ds_squared = conformal_factor * (-(dy_dt**2) + (dx_dt**2))
20        total_interval += math.sqrt(ds_squared) * dt # no abs(): relies on
        the causal filter
    return total_interval

```

Listing 8: Causally filtered grid search, maximum (code/08_ads_grid_filtered.py).

```

1 import math
2 import numpy as np
3
4 def generate_ads_inflection_heatmap(start_x, start_y, end_x, end_y, Lambda
   =1.0,
5                                     x_lim=3.0, resolution=80):
6     x_coords = np.linspace(-x_lim, x_lim, resolution)
7     margin = 0.05
8     y_coords = np.linspace(-math.pi / 2 + margin, math.pi / 2 - margin,
9                             resolution)
10    heatmap = np.full((resolution, resolution), np.nan)
11
12    for i, y in enumerate(y_coords):
13        for j, x in enumerate(x_coords):
14            # Geometric filter: 45-degree causal constraints
15            from_start_wedge = (x - start_x) >= abs(y - start_y)
16            to_end_wedge = (end_x - x) >= abs(y - end_y)
17            if from_start_wedge and to_end_wedge:
18                dist1 = integrate_ads_segment(start_x, start_y, x, y, Lambda=
19                    Lambda)
20                dist2 = integrate_ads_segment(x, y, end_x, end_y, Lambda=
21                    Lambda)
22                heatmap[i, j] = dist1 + dist2
23    return x_coords, y_coords, heatmap
24
25 start_x, start_y = 0.0, 0.0
26 end_x, end_y = np.pi / 2, np.pi / 4
27
28 x_vals, y_vals, dist_grid = generate_ads_inflection_heatmap(
29     start_x, start_y, end_x, end_y, Lambda=1.0, x_lim=3.0, resolution=800)
30
31 max_idx = np.unravel_index(np.nanargmax(dist_grid), dist_grid.shape)
32 opt_y_grid = y_vals[max_idx[0]]
33 opt_x_grid = x_vals[max_idx[1]]
34 max_dist_grid = dist_grid[max_idx]
35 # (1.0476, 0.6491), maximum 1.5585

```

For start (0,0) and end $(\pi/2, \pi/4)$:

Method	Inflection point (ρ, η)	Path value
Grid search (800×800)	(1.0476, 0.6491)	1.5585
Grid search (300×300)	(1.0535, 0.6561)	1.5585
Restricted to the exact geodesic	(1.0486, 0.6597)	1.5584
Newton–Raphson (Section 4.5)	(1.0473, 0.6505)	1.5586

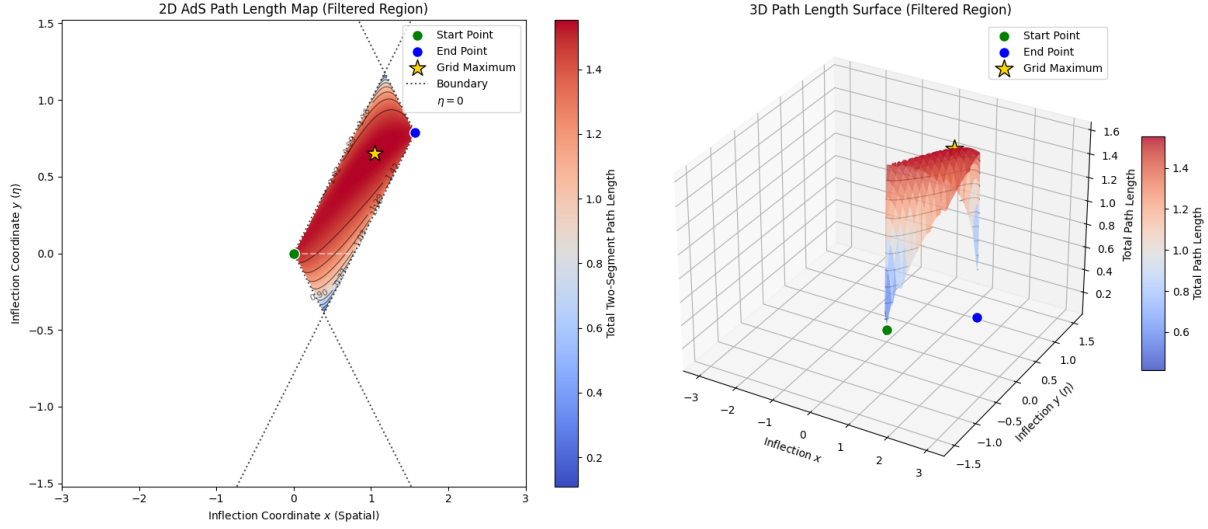


Figure 8: After the fix: causally filtered heatmap and 3D surface, with the maximum at $(1.0476, 0.6491)$.

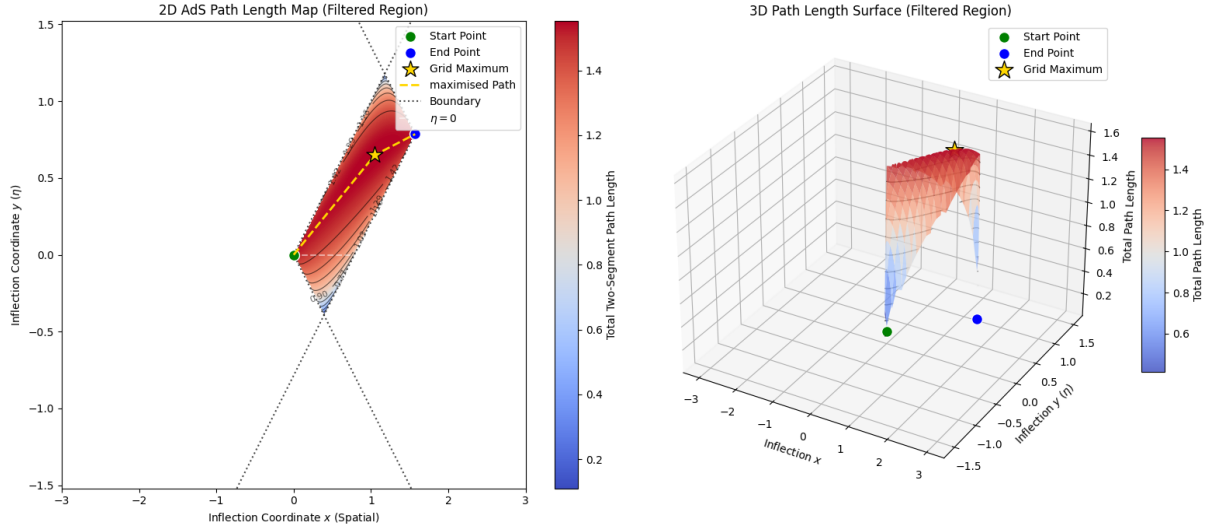


Figure 9: The maximising two-segment path drawn on the heatmap (start, grid maximum, end).

The maximising two-segment path appears to follow the geodesic, which is what we would expect if it is the best two-segment approximation to it. At this point that is only an impression from the pictures, and the natural next question is how to test it. That needs the geodesic itself, computed independently.

3.4 Geodesic solver

To test the claim we need the geodesic from a method that does not know about our piecewise paths. Our first attempt used the Lagrange (Euler–Lagrange) equations, which generated the paths we expected for some pairs of points and failed for others. That pushed us to a more robust approach: solve the geodesic equations of the conformal AdS metric as a boundary value problem (SciPy `solve_bvp`), fixing the start and end points and starting from a straight-line guess.

Listing 9: Geodesic by boundary value problem (code/09_geodesic_bvp.py).

```

1 import numpy as np
2 from scipy.integrate import solve_bvp
3
4 def ads_geodesic_derivs(t, U):
5     eta, rho, omega, v_eta, v_rho, v_omega = U
6     tan_eta = np.tan(eta)
7     sin_rho = np.sin(rho)
8     cos_rho = np.cos(rho)
9     cot_rho = np.where(np.abs(sin_rho) < 1e-5, np.sign(sin_rho) * 1e5,
10                        cos_rho / np.where(np.abs(sin_rho) < 1e-5, 1e-5,
11                                           sin_rho))
12
13     deta_dt = v_eta
14     drho_dt = v_rho
15     domega_dt = v_omega
16     dv_eta_dt = -tan_eta * (v_eta**2 + v_rho**2 + (sin_rho**2) * v_omega**2)
17     dv_rho_dt = -2 * v_rho * v_eta * tan_eta + sin_rho * cos_rho * v_omega**2
18     dv_omega_dt = -2 * v_omega * v_rho * cot_rho - 2 * v_omega * v_eta *
19                    tan_eta
20
21     return np.vstack((deta_dt, drho_dt, domega_dt, dv_eta_dt, dv_rho_dt,
22                       dv_omega_dt))
23
24 def solve_ads_geodesic(start_x, start_y, end_x, end_y, num_points=500):
25     def boundary_conditions(U_start, U_end):
26         return np.array([
27             U_start[0] - start_y, U_start[1] - start_x, U_start[2] - 0.0,
28             U_end[0] - end_y, U_end[1] - end_x, U_end[2] - 0.0,
29         ])
30
31     t_mesh = np.linspace(0, 1, 200)
32     guess_eta = np.linspace(start_y, end_y, 200)
33     guess_rho = np.linspace(start_x, end_x, 200)
34     zeros = np.zeros(200)
35     U_guess = np.vstack((guess_eta, guess_rho, zeros, zeros, zeros, zeros))
36
37     res = solve_bvp(ads_geodesic_derivs, boundary_conditions, t_mesh, U_guess,
38                    max_nodes=5000)
39
40     if res.success:
41         t_final = np.linspace(0, 1, num_points)
42         sol = res.sol(t_final)
43         return sol[1], sol[0], t_final # rho (x), eta (y), parameter t
44     return None, None, None

```

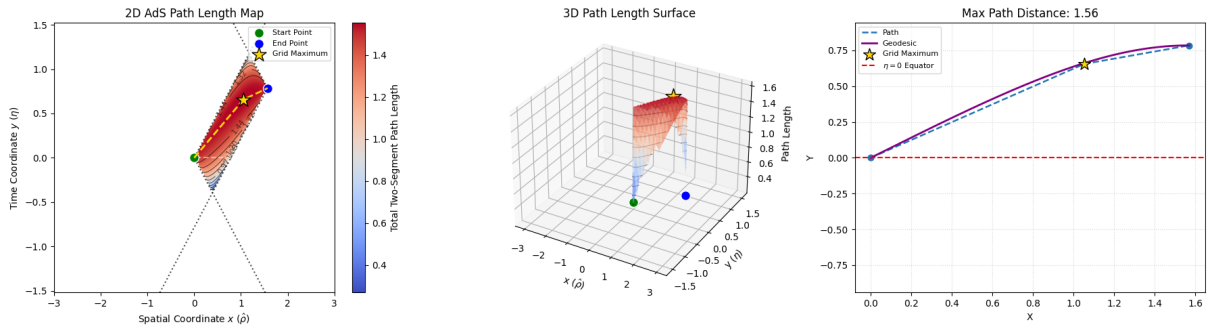


Figure 10: Grid maximum compared with the exact geodesic from the boundary value solver (right panel).

Overlaying the two shows close agreement. A sharper test is available, because we now have a curve to restrict to. If the geodesic really is where the best corner belongs, then searching only along the geodesic (parameterised by $t \in [0, 1]$) should find the same corner as searching the whole plane. It does: the path length along the geodesic has a single smooth peak at $t \approx 0.565$, with value 1.5584, against 1.5585 from the full grid search.

Listing 10: Optimising along the geodesic only (code/10_geodesic_search.py).

```

1 import numpy as np
2
3 def evaluate_inflection_along_geodesic(geo_x, geo_y, start_x, start_y, end_x,
4   end_y,
5   Lambda=1.0):
6     """Two-segment length for every point of the geodesic used as inflection
7     point."""
8     distances = []
9     for x, y in zip(geo_x, geo_y):
10         dist1 = integrate_ads_segment(start_x, start_y, x, y, Lambda=Lambda)
11         dist2 = integrate_ads_segment(x, y, end_x, end_y, Lambda=Lambda)
12         distances.append(dist1 + dist2)
13     distances = np.array(distances)
14
15     # Exclude the end points: there the path degenerates to a single segment
16     inner_mask = np.ones(len(distances), dtype=bool)
17     inner_mask[0] = False
18     inner_mask[-1] = False
19     max_idx = np.argmax(distances[inner_mask])
20     return distances, geo_x[max_idx], geo_y[max_idx], distances[max_idx],
21     max_idx
22
23 # geo_x, geo_y, geo_t = solve_ads_geodesic(0.0, 0.0, np.pi/2, np.pi/4)
24 # -> optimum (1.0486, 0.6597), 1.5584, at t = 0.5651

```

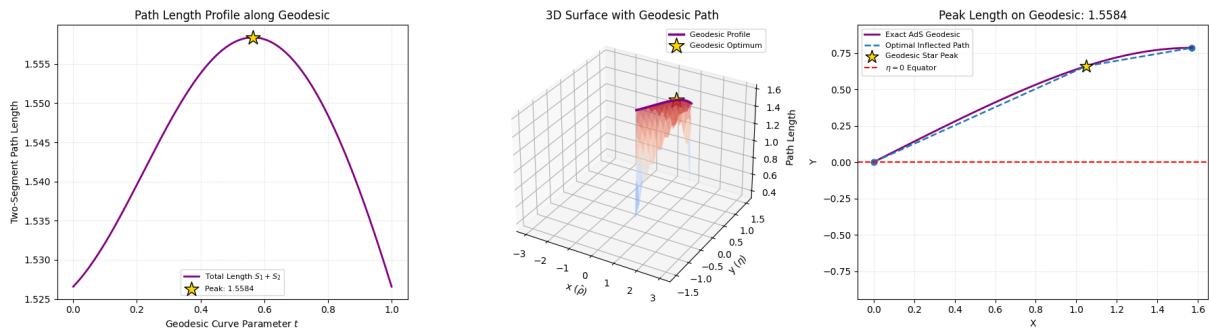


Figure 11: Search restricted to the geodesic: path length profile along t , 3D surface and the optimal inflected path. Peak 1.5584 at $t = 0.5651$.

3.5 Recursive reconstruction of geodesics

Since the best two-segment path hugs the geodesic, a natural question is whether the geodesic can be rebuilt from this one step alone. If the best corner between two points lies on the geodesic, then repeating the step between the start and that corner, and between the corner and the end, and recursing, should give finer and finer approximations. For the geodesically connected test pair, $n = 5$ levels of subdivision (33 nodes) produce a path that sits very close to the exact geodesic.

Listing 11: Recursive subdivision (code/11_recursive_reconstruction.py).

```

1 import math
2 import numpy as np
3
4 def find_optimal_midpoint(xa, ya, xb, yb, Lambda=1.0, grid_res=60):
5     t_vals = np.linspace(0.1, 0.9, grid_res)
6     y_vals = np.linspace(-math.pi / 2 + 0.1, math.pi / 2 - 0.1, grid_res)
7
8     best_dist = -1.0
9     best_pt = ((xa + xb) / 2.0, (ya + yb) / 2.0)
10
11     for t in t_vals:
12         x_c = xa + t * (xb - xa)
13         for y_c in y_vals:
14             # Causal cone validation
15             if (x_c - xa) >= abs(y_c - ya) and (xb - x_c) >= abs(yb - y_c):
16                 d1 = integrate_ads_segment(xa, ya, x_c, y_c, Lambda=Lambda)
17                 d2 = integrate_ads_segment(x_c, y_c, xb, yb, Lambda=Lambda)
18                 total_d = d1 + d2
19                 if total_d > best_dist and not math.isinf(total_d):
20                     best_dist = total_d
21                     best_pt = (x_c, y_c)
22     return best_pt
23
24
25 def reconstruct_geodesic_n_level(pts, n, Lambda=1.0):
26     """Recursively subdivide the path n times (n = 5 gives 33 points)."""
27     if n == 0 or len(pts) < 2:
28         return pts
29     for _ in range(n):
30         new_pts = [pts[0]]
31         for i in range(len(pts) - 1):
32             p1, p2 = pts[i], pts[i + 1]
33             mid = find_optimal_midpoint(p1[0], p1[1], p2[0], p2[1], Lambda=
34                                         Lambda)
35             new_pts.extend([mid, p2])
36         pts = new_pts
37     return pts
38
39 # Figure 13: initial_path = [(0, 0), (pi/2, pi/4)], n = 5
40 # Figure 14: initial_path = [(-3, 0), (3*pi/4, pi/4)], n = 5

```

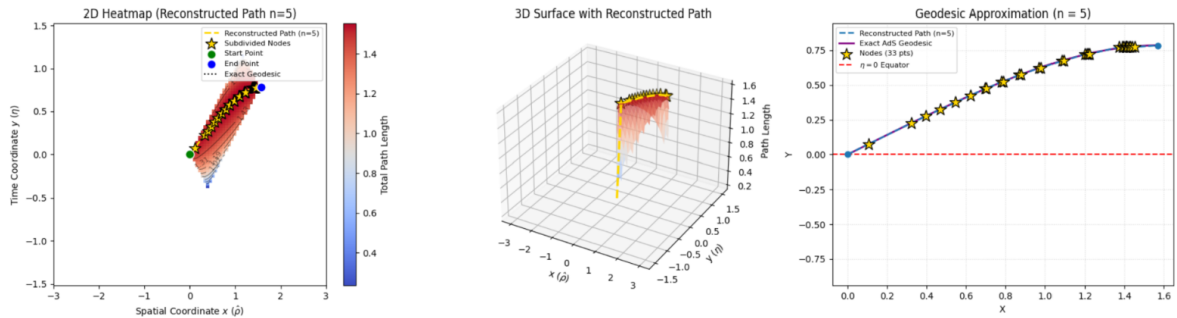


Figure 12: Recursive reconstruction ($n = 5$, 33 nodes) for the geodesically connected pair: the nodes lie on the exact geodesic.

The obvious follow-up is whether the same idea works when the points are not connected by a geodesic, and so far it does not. For a start at $(-3, 0)$ and an end at $(3\pi/4, \pi/4)$, the reconstructed nodes along $\eta \approx 1.47$ and sit on the search boundary (the code limits η to

within 0.1 of $\pi/2$), far from the geodesic returned by the solver. In this case the result reflects the bounds of the search rather than a true extremal path.

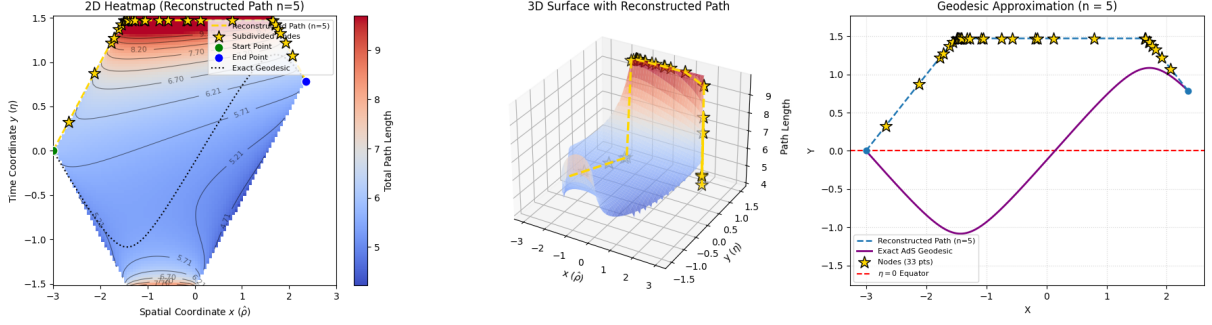


Figure 13: Failure case: start $(-3, 0)$, end $(3\pi/4, \pi/4)$. The reconstructed path hugs the search boundary near $\eta = 1.47$, far from the geodesic.

Because the failed path piles up against the boundary and bends sharply, one fix we thought of is to penalise sharp corners, maximising proper time minus a penalty on the deflection angle at each corner, but we did not get time to try this.

4 Analytic work

$$(\eta_i, \rho_i) \xrightarrow{S_1} (\eta_s, \rho_s) \xrightarrow{S_2} (\eta_f, \rho_f)$$

The numerics tell us where the maximum is, but not why it is there. Each straight segment has a closed-form interval, so we can differentiate the total interval and solve for the extremum directly. We first need that interval, so the first step is to find the distance between two points joined by a straight line.

4.1 Interval along a straight line

Start from the line element,

$$ds^2 = \frac{-d\eta^2 + d\rho^2}{\Lambda^2 \cos^2 \eta} \quad \Rightarrow \quad ds = \frac{\sqrt{-d\eta^2 + d\rho^2}}{\Lambda \cos \eta}, \quad S = \int_{\gamma} \frac{\sqrt{-d\eta^2 + d\rho^2}}{\Lambda \cos \eta}. \quad (1)$$

For a straight line the path γ can be parametrised as

$$\eta(t) = \eta_i + t \Delta\eta, \quad \Delta\eta = \eta_f - \eta_i, \quad t \in (0, 1), \quad (2)$$

$$\rho(t) = \rho_i + t \Delta\rho, \quad \Delta\rho = \rho_f - \rho_i, \quad (3)$$

so that

$$\frac{d\eta}{dt} = \Delta\eta \Rightarrow d\eta = \Delta\eta dt, \quad \frac{d\rho}{dt} = \Delta\rho \Rightarrow d\rho = \Delta\rho dt. \quad (4)$$

Therefore

$$S = \int_0^1 \frac{\sqrt{-(\Delta\eta)^2 + \Delta\rho^2}}{\Lambda \cos(\eta_i + t \Delta\eta)} dt = \frac{\sqrt{-(\Delta\eta)^2 + \Delta\rho^2}}{\Lambda} \int_0^1 \frac{dt}{\cos(\eta_i + t \Delta\eta)}. \quad (5)$$

The remaining integral is a standard one. Substituting $u = \eta_i + t \Delta\eta$, so that $dt = du/\Delta\eta$, and using $\int \sec u du = \ln \left| \tan \left(\frac{u}{2} + \frac{\pi}{4} \right) \right|$,

$$\int_0^1 \frac{dt}{\cos(\eta_i + t \Delta\eta)} = \frac{1}{\Delta\eta} \ln \left| \frac{\tan \left(\frac{\eta_i + \Delta\eta}{2} + \frac{\pi}{4} \right)}{\tan \left(\frac{\eta_i}{2} + \frac{\pi}{4} \right)} \right|. \quad (6)$$

Since $\eta_i + \Delta\eta = \eta_f$, the interval along a straight line is

$$S = \frac{\sqrt{-(\Delta\eta)^2 + \Delta\rho^2}}{\Lambda \Delta\eta} \ln \left| \frac{\tan\left(\frac{\eta_f}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_i}{2} + \frac{\pi}{4}\right)} \right|. \quad (7)$$

This is for a spacelike segment, $\Delta\rho^2 > \Delta\eta^2$. For a horizontal segment ($\Delta\eta \rightarrow 0$) the logarithm tends to $\Delta\eta \sec \eta$ and the expression tends to $|\Delta\rho| \sec \eta / \Lambda$.

4.2 Total interval and the extremum conditions

For a path from the start point (η_i, ρ_i) to the inflection point (η_s, ρ_s) and on to the end point (η_f, ρ_f) , applying (7) to each segment gives the total interval

$$S_T(\eta_s, \rho_s) = S_1 + S_2 = \frac{\sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}}{\Lambda(\eta_s - \eta_i)} \ln \left| \frac{\tan\left(\frac{\eta_s}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_i}{2} + \frac{\pi}{4}\right)} \right| + \frac{\sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}}{\Lambda(\eta_f - \eta_s)} \ln \left| \frac{\tan\left(\frac{\eta_f}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_s}{2} + \frac{\pi}{4}\right)} \right|. \quad (8)$$

From the numerical work, this function is to be maximised over the inflection point, i.e.

$$\frac{\partial S_T}{\partial \rho_s} = 0 \quad (1) \quad \text{and} \quad \frac{\partial S_T}{\partial \eta_s} = 0 \quad (2). \quad (9)$$

4.3 Condition (1): derivative with respect to ρ_s

We take condition (1) first because it is the easier of the two: the logarithmic factors depend only on the η coordinates, so they behave as constants when we vary ρ_s .

$$\frac{\partial S_T}{\partial \rho_s} = \frac{\partial S_1}{\partial \rho_s} + \frac{\partial S_2}{\partial \rho_s}. \quad (10)$$

For the first segment,

$$S_1 = \frac{\sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}}{\Lambda(\eta_s - \eta_i)} \ln \left| \frac{\tan\left(\frac{\eta_s}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_i}{2} + \frac{\pi}{4}\right)} \right|. \quad (11)$$

Let

$$c_1 = \frac{1}{\Lambda(\eta_s - \eta_i)} \ln \left| \frac{\tan\left(\frac{\eta_s}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_i}{2} + \frac{\pi}{4}\right)} \right|, \quad (12)$$

which does not depend on ρ_s . Then

$$S_1 = c_1 \sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}, \quad \frac{\partial S_1}{\partial \rho_s} = \frac{c_1(\rho_s - \rho_i)}{\sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}}. \quad (13)$$

For the second segment,

$$S_2 = \frac{\sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}}{\Lambda(\eta_f - \eta_s)} \ln \left| \frac{\tan\left(\frac{\eta_f}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_s}{2} + \frac{\pi}{4}\right)} \right|. \quad (14)$$

Let

$$c_2 = \frac{1}{\Lambda(\eta_f - \eta_s)} \ln \left| \frac{\tan\left(\frac{\eta_f}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_s}{2} + \frac{\pi}{4}\right)} \right|, \quad (15)$$

which also does not depend on ρ_s . Then

$$S_2 = c_2 \sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}, \quad \frac{\partial S_2}{\partial \rho_s} = \frac{-c_2(\rho_f - \rho_s)}{\sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}}. \quad (16)$$

Setting $\frac{\partial S_1}{\partial \rho_s} + \frac{\partial S_2}{\partial \rho_s} = 0$ gives

$$\frac{c_1(\rho_s - \rho_i)}{\sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}} = \frac{c_2(\rho_f - \rho_s)}{\sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}}. \quad (17)$$

Each side contains the ratio of a spatial increment to the square root of (spatial increment)²–(time increment)², which is at least 1 for a spacelike segment. Writing this ratio as a hyperbolic cosine,

$$\cosh \phi_1 = \frac{\rho_s - \rho_i}{\sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}}, \quad \cosh \phi_2 = \frac{\rho_f - \rho_s}{\sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}}, \quad (18)$$

equation (17) becomes

$$\boxed{c_1 \cosh \phi_1 = c_2 \cosh \phi_2} \quad (19)$$

At this stage it was not obvious whether this relation is useful on its own: it constrains ρ_s only for a given η_s , and we still need condition (2). It does, however, say that one particular combination is the same on the two sides of the kink, which looks like a conservation law, so we come back to it in the remarks at the end of this section.

4.4 Condition (2): derivative with respect to η_s

This one is harder, because η_s appears both in the square-root prefactor and inside the logarithm of each segment. Each segment is therefore a product of two functions of η_s , and we need the product rule.

$$\frac{\partial S_T}{\partial \eta_s} = \frac{\partial S_1}{\partial \eta_s} + \frac{\partial S_2}{\partial \eta_s}. \quad (20)$$

Now η_s appears in both factors of S_1 , so use the product rule. Let

$$f(\eta_s) = \frac{\sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}}{\Lambda(\eta_s - \eta_i)}, \quad g(\eta_s) = \ln \left| \frac{\tan\left(\frac{\eta_s}{2} + \frac{\pi}{4}\right)}{\tan\left(\frac{\eta_i}{2} + \frac{\pi}{4}\right)} \right|. \quad (21)$$

Since $S_1 = f(\eta_s)g(\eta_s)$,

$$\frac{dS_1}{d\eta_s} = f'(\eta_s)g(\eta_s) + f(\eta_s)g'(\eta_s). \quad (22)$$

The derivative of f . Let $\eta_s - \eta_i = u$ and $\rho_s - \rho_i = k$. Then

$$f = \frac{\sqrt{-u^2 + k^2}}{\Lambda u} = \frac{\sqrt{k^2 - u^2}}{\Lambda u}. \quad (23)$$

By the quotient rule,

$$\frac{df}{du} = \frac{1}{\Lambda} \frac{\left(\frac{-u}{\sqrt{k^2 - u^2}}\right)(u) - \sqrt{k^2 - u^2}(1)}{u^2} = \frac{1}{\Lambda} \frac{-k^2}{u^2 \sqrt{k^2 - u^2}}, \quad (24)$$

and so

$$f'(\eta_s) = \frac{-(\rho_s - \rho_i)^2}{\Lambda(\eta_s - \eta_i)^2 \sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}}. \quad (25)$$

The derivative of g .

$$g'(\eta_s) = \sec \eta_s. \quad (26)$$

(With $\theta = \frac{\eta_s}{2} + \frac{\pi}{4}$: $g' = \frac{1}{2} \sec^2 \theta / \tan \theta = 1 / \sin 2\theta = 1 / \sin(\eta_s + \frac{\pi}{2}) = \sec \eta_s$.)

Result for the first segment.

$$\frac{\partial S_1}{\partial \eta_s} = \frac{-(\rho_s - \rho_i)^2}{\Lambda(\eta_s - \eta_i)^2 \sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}} \ln \left| \frac{\tan(\frac{\eta_s}{2} + \frac{\pi}{4})}{\tan(\frac{\eta_i}{2} + \frac{\pi}{4})} \right| + \frac{\sqrt{-(\eta_s - \eta_i)^2 + (\rho_s - \rho_i)^2}}{\Lambda(\eta_s - \eta_i)} \sec \eta_s. \quad (27)$$

Result for the second segment. The same steps apply to S_2 , with $\eta_f - \eta_s$ in place of $\eta_s - \eta_i$. Because η_s now enters with the opposite sign, $d(\eta_f - \eta_s)/d\eta_s = -1$, the derivative of the logarithm is $-\sec \eta_s$ and the derivative of the prefactor changes sign:

$$\frac{\partial S_2}{\partial \eta_s} = \frac{(\rho_f - \rho_s)^2}{\Lambda(\eta_f - \eta_s)^2 \sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}} \ln \left| \frac{\tan(\frac{\eta_f}{2} + \frac{\pi}{4})}{\tan(\frac{\eta_s}{2} + \frac{\pi}{4})} \right| - \frac{\sqrt{-(\eta_f - \eta_s)^2 + (\rho_f - \rho_s)^2}}{\Lambda(\eta_f - \eta_s)} \sec \eta_s. \quad (28)$$

Condition (2) is the sum of (27) and (28) set to zero.

4.5 Solution by multivariate Newton–Raphson

Setting $\partial S_T / \partial \eta_s = 0$ looks hard to solve by hand, since η_s appears inside both the prefactor and the logarithms. The two conditions do, however, give us an independent check on the grid search if we solve them numerically. Both are equations in the same two unknowns, so they can be solved together numerically with a multivariate Newton–Raphson iteration. The iteration is

$$\mathbf{x}_{n+1} = \mathbf{x}_n - H^{-1}(\mathbf{x}_n) \nabla S_T(\mathbf{x}_n), \quad \mathbf{x} = (\eta_s, \rho_s), \quad (29)$$

with H the Hessian of S_T . The test case is $(\eta_i, \rho_i) = (0, 0)$ and $(\eta_f, \rho_f) = (\pi/4, \pi/2)$, with $\Lambda = 1$ and initial guess $(\eta_s, \rho_s) = (0.392699, 0.785398)$. The early iterates wander (several leave the physical range $|\eta| < \pi/2$), after which the iteration converges in 17 steps, with the gradient norm falling from 4.9×10^{-1} at the first iteration to 4.6×10^{-10} at the last.

Quantity	Value
Optimal η_s	0.650471
Optimal ρ_s	1.047288
Maximum S_T	1.558551
$\partial^2 S_T / \partial \eta_s^2$	-4.071223
$\det H$	0.743144

Since $H_{\eta\eta} < 0$ and $\det H > 0$, the Hessian is negative definite and the point is a confirmed local maximum. This matches the numerical simulation almost perfectly, so the analytic conditions and the grid search are describing the same point: the grid-search maximum is $(\rho, \eta) = (1.0476, 0.6491)$ and the geodesic-constrained optimum is $(1.0486, 0.6597)$.

4.6 Further remarks

4.6.1 The constants c_k as averages of the conformal factor

Write the conformal factor in the line element as $\Omega(\eta) = 1/(\Lambda \cos \eta)$. For a segment from η_a to η_b with $R = \sqrt{\Delta \rho^2 - \Delta \eta^2}$, the interval (7) is $S = cR$ with

$$c = \frac{1}{\Lambda \Delta \eta} \ln \left| \frac{\tan(\frac{\eta_b}{2} + \frac{\pi}{4})}{\tan(\frac{\eta_a}{2} + \frac{\pi}{4})} \right| = \frac{1}{\Delta \eta} \int_{\eta_a}^{\eta_b} \Omega(\eta) d\eta. \quad (30)$$

So c is the average of the conformal factor over the segment, i.e. the interval per unit flat length. This is the sense in which c_1 and c_2 play the role of refractive indices below.

4.6.2 Rapidity and the hyperbolic form of condition (2)

For a spacelike segment, $\Delta\rho_k^2 - \Delta\eta_k^2 = R_k^2 > 0$, so there is an angle ϕ_k (the rapidity, or boost angle, of the segment) with

$$\Delta\rho_k = R_k \cosh \phi_k, \quad \Delta\eta_k = R_k \sinh \phi_k, \quad \tanh \phi_k = \frac{\Delta\eta_k}{\Delta\rho_k}, \quad (31)$$

where $\Delta\rho_1 = \rho_s - \rho_i$, $\Delta\eta_1 = \eta_s - \eta_i$, $\Delta\rho_2 = \rho_f - \rho_s$, $\Delta\eta_2 = \eta_f - \eta_s$. This is the ϕ_k used in (19). Writing $\Omega_s = \sec \eta_s / \Lambda$ and using $\ln |\cdots|_k = \Lambda \Delta\eta_k c_k$, the derivatives (27) and (28) simplify to

$$\frac{\partial S_1}{\partial \eta_s} = \frac{\Omega_s - c_1 \cosh^2 \phi_1}{\sinh \phi_1}, \quad \frac{\partial S_2}{\partial \eta_s} = \frac{c_2 \cosh^2 \phi_2 - \Omega_s}{\sinh \phi_2}, \quad (32)$$

so condition (2) is

$$\frac{\Omega_s - c_1 \cosh^2 \phi_1}{\sinh \phi_1} = \frac{\Omega_s - c_2 \cosh^2 \phi_2}{\sinh \phi_2}. \quad (33)$$

Once condition (1) holds, $c_k \cosh^2 \phi_k = P \cosh \phi_k$ with $P = c_1 \cosh \phi_1 = c_2 \cosh \phi_2$, and this becomes $(\Omega_s - P \cosh \phi_1) / \sinh \phi_1 = (\Omega_s - P \cosh \phi_2) / \sinh \phi_2$.

4.6.3 Snell's law and momentum

Equation (19) is very similar to Snell's law, $n_1 \sin \theta_1 = n_2 \sin \theta_2$, with the segment averages c_k playing the role of refractive indices (so one may equally write $n_1 \cosh \phi_1 = n_2 \cosh \phi_2$). The analogy is structural rather than exact: $\cosh \phi$ is the hyperbolic counterpart of $\sin \theta$, because the geometry is Lorentzian.

The reason for the resemblance is plausibly that the optimisation condition is a least-action principle. For a path $(\eta(\lambda), \rho(\lambda))$ the interval is $S = \int L d\lambda$ with $L = \Omega(\eta) \sqrt{\rho'^2 - \eta'^2}$, and L does not depend on ρ is invariant under ρ -translations). Along a true geodesic the conjugate momentum is therefore conserved:

$$p_\rho = \frac{\partial L}{\partial \rho'} = \Omega(\eta) \frac{\rho'}{\sqrt{\rho'^2 - \eta'^2}} = \Omega(\eta) \cosh \phi = \text{const}. \quad (34)$$

For the piecewise path, Ω is replaced on each segment by its average c_k , and (19) says that this ρ -momentum is the same on either side of the kink. It is therefore the discrete analogue of conservation of p_ρ , which suggests the link to momentum noted in the original working. (The geodesic solver includes an angular coordinate ω , but with ω held fixed the motion stays in the (η, ρ) plane, where this reduction applies.) As the subdivision becomes finer, $c_k \rightarrow \Omega(\eta)$ and the condition tends to the statement that $\Omega \cosh \phi$ is constant along the geodesic. Condition (33) is the additional constraint that fixes the height η_s of the kink.

5 Open questions and next steps

1. **Points not connected by a geodesic.** The recursive reconstruction only works so far where a geodesic exists, and the Euler–Lagrange approach also failed for some pairs. The natural question is what the extremal piecewise path looks like in the other case, and whether the same logic applies.

2. **Feynman (path-integral) approach.** This was floated at the last meeting as a way to handle exactly those pairs. It would be new to me, and I would welcome guidance on whether it is the right direction or whether a more efficient numerical method for geodesics exists.
3. **Analytical side.** The conditions (17) and (33) can be plotted over the (ρ_s, η_s) plane and compared with the heatmaps. The Snell's-law-like relation (19) and its momentum interpretation are worth investigating further.
4. **Corner-penalised subdivision.** The drafted penalised search should be run, to see whether it removes the boundary-hugging behaviour in the failure case.